Assignment 3: Time-Series Data

Shivam Shekher · Advanced Machine Learning · November 2026

Executive Summary

I used recurrent neural networks to forecast the air temperature in Jena, Germany, 24 hours ahead, using the previous five days of weather readings. Before trying anything new, I re-ran the four models from class with my own pipeline, so I had my own numbers to compare against. Then I tried the three improvements the assignment suggests: changing the number of units in the stacked GRU, replacing the GRU layers with LSTM layers, and putting 1D convolution layers in front of the RNN. That came to 11 models. I compared them all on validation MAE, and the three best were run once on the test set.

The stacked GRU came out on top. The version with 64 and 32 units had the lowest validation MAE (2.226 °C, 8.9% better than the common-sense baseline of 2.444 °C). On the test years, the three finalists scored between 2.374 and 2.413 °C against 2.622 °C for the baseline, which is 8.0 to 9.5% better. Changing the number of units hardly mattered: all four GRU sizes ended within 0.03 °C of each other, which is about the noise I saw between repeat runs. The LSTM was clearly worse than the GRU at the same size, and the Conv1D + RNN models trained more than twice as fast but were less accurate.

I recommend the stacked GRU with 64/32 units, because it is the model my selection rule picked on validation MAE. The three GRU finalists are tied within the noise, so the class's 32/32 version is just as good a choice if a smaller model is wanted. It has less than half the parameters.

Key Terms

Term	What it means here
Validation MAE	Mean absolute error on the validation years (2013–2014), in °C: on average, how many degrees the 24-hour forecast is off. Lower is better. Every choice between models was made on this number.
Common-sense baseline	A forecast with no learning at all: the temperature in 24 hours will be the same as it is now. It scores 2.444 °C on validation and 2.622 °C on test, so a model has to beat these numbers to be worth using.
Training, validation and test periods	The data is split by time, not at random: 2009–2012 to train, 2013–2014 to compare models and decide when to stop training, and 2015–2016 for the final test. The model never learns from the future.
Window (sample)	One input to the network: 120 hourly readings of all 14 weather variables, which is 5 days. The target is the temperature 24 hours after the last reading.
GRU and LSTM	Two kinds of recurrent layer. Both read the window one hour at a time and keep a memory of what they have seen, and their gates decide what to keep and what to forget. The LSTM has an extra gate and a separate memory track, so it has more weights.
Stacked setup	Two recurrent layers on top of each other, where the first passes its whole output sequence to the second. "64/32" means 64 units in the first layer and 32 in the second.
Recurrent dropout	Dropout applied to the layer's memory connections, with the same mask at every time step (Gal and Ghahramani, 2016). It is how the class models avoid overfitting.
Conv1D + pooling	A 1D convolution slides a short filter (6 hours here) along the time axis and reacts to local patterns. Max-pooling then halves the length of the sequence.

Term	What it means here
Early stopping	Training stops when validation MAE has not improved for 3 epochs, and the weights from the best epoch are put back.

1. How I Ran the Experiments

The data is the Jena Climate dataset, recorded by the Max Planck Institute for Biogeochemistry. It has 14 weather variables, such as air temperature, pressure, humidity and wind, measured every 10 minutes. Like the course, I used the years 2009 to 2016, from the copy hosted by TensorFlow, which has 420,551 rows. I wrote and debugged the notebook in Google Colab, but my free GPU allowance ran out partway through, so the final run in this report was done in a Kaggle notebook. It used the same Tesla T4 GPU and the same TensorFlow version (2.21.0).

Cleaning the data. I checked the file before training anything. 327 timestamps appear twice, and every one of them is an exact copy of the row before, so I removed them, which left 420,224 rows. Wind speed had 38 values of −9999 (18 in wv and 20 in max. wv). That is an error code, not a real speed, so I set them to 0, as the TensorFlow time series tutorial does. The tutorial's own code for this step no longer changes the data with current pandas, so I used .loc instead and checked that the minimum is now 0. The readings are also not perfectly regular: there are five gaps longer than 10 minutes. The longest is 3 days starting on 25 October 2016. I kept the gaps, as the course does.

Splitting and windows. I split the rows by time: the first half for training, the next quarter for validation and the last quarter for testing (Table 1). Each variable was standardized with the mean and standard deviation of the training rows only, so nothing about the later years leaks into training. Each sample is a window of 120 hourly steps (every sixth reading), and the target is the temperature 24 hours after the last step. These are exactly the samples that keras.utils.timeseries_dataset_from_array gives in class. I checked my version against it window by window, but I cut each batch with a single gather, which turned out to be much faster and to use far less memory. With this pipeline the common-sense baseline scored 2.444 °C on validation and 2.622 °C on test, the same as the course (2.44 and 2.62), so I was confident the setup matched the one from class.

Split	Period	Rows	Windows
Training	1 Jan 2009 – 30 Dec 2012	210,112	209,398
Validation	30 Dec 2012 – 30 Dec 2014	105,056	104,342
Test	30 Dec 2014 – 1 Jan 2017	105,056	103,484

Table 1. How the 420,224 cleaned rows were split. A window needs 120 hourly steps, so each split has 714 fewer windows than rows. The test set loses another 858 because its last rows have no temperature 24 hours later.

The models. I started with the four models from class: a dense network, an LSTM with 16 units, an LSTM with 32 units and recurrent dropout, and the stacked GRU with 32 units in each layer. The stacked GRU is the base for the three methods. For Method 1 I changed only the number of units: smaller (16/16), larger (64/64), and a funnel with a wider first layer (64/32). For Method 2 I replaced the GRU layers with LSTM layers, at 32/32 and at the best GRU size from Method 1. The notebook picked that size automatically from the validation results, and it came out as 64/32. For Method 3 I put two Conv1D layers (32 filters, a kernel of 6 hours) with max-pooling between them in front of a single recurrent layer of 32 units, once with a GRU and once with an LSTM. The pooling halves the window from 120 to 60 steps, and the recurrent layer then

reads it in order. All the models with dropout used the class settings: recurrent dropout of 0.5 (0.25 for the single LSTM) and dropout of 0.5 before the output.

Training settings. Every model was trained the same way: RMSprop with a learning rate of 0.001, mean squared error as the loss, MAE as the metric, batches of 256 windows and at most 30 epochs. Early stopping ended training after 3 epochs without a better validation MAE and restored the best weights, and the learning rate was halved after 2 epochs without improvement. The seed was 42 for every model. The test set was used once, at the very end, for the three models with the lowest validation MAE. I fixed that rule before seeing any test result.

Making it run. Two practical problems took more time than the models did. First, recurrent dropout switches off the fast cuDNN version of the GRU and LSTM layers (Keras documentation), and one epoch of a small GRU went from 29 seconds to 310. Unrolling the loop over the 120 steps (unroll=True) brought it back to 30 seconds without changing the model's equations. Second, Colab kept running out of memory. TensorFlow held on to 0.6–2 GB after every model, and after a few models the session was using 9.3 of its 12.7 GB. I fixed this by training each model in its own Python process, which gives all of its memory back when it finishes.

ID	Model	Method	Parameters	Why I ran it
1	Dense (Flatten, Dense 16)	Course reference	26,913	First learned model in class
2	LSTM(16)	Course reference	2,001	First recurrent model in class
3	LSTM(32) + recurrent dropout	Course reference	6,049	Class model with dropout
4	Stacked GRU 32/32	Course reference	10,977	Best class model; base for Methods 1–3
5	Stacked GRU 16/16	Method 1: units	3,185	Fewer units
6	Stacked GRU 64/64	Method 1: units	40,385	More units
7	Stacked GRU 64/32	Method 1: units	24,801	Wider first layer
8	Stacked LSTM 32/32	Method 2: LSTM	14,369	LSTM at the class size
9	Stacked LSTM 64/32	Method 2: LSTM	32,673	LSTM at the best GRU size
10	Conv1D + GRU 32	Method 3: Conv1D + RNN	15,265	Convolutions before a GRU
11	Conv1D + LSTM 32	Method 3: Conv1D + RNN	17,249	Convolutions before an LSTM

Table 2. The 11 models. All of them use the same windows, seed, optimizer and callbacks, so only the architecture changes.

2. What I Found

The course models behaved the way they did in class. The dense network (2.481 °C) did worse than the baseline, and the LSTM(16) without dropout was only 1.2% better than it (2.415 °C). Neither improved for long: the dense model peaked at epoch 2, the LSTM(16) hardly moved after epoch 2, and at their best epoch both already had a lower training MAE than validation MAE (2.331 against 2.481 °C for the dense model). That is overfitting. Adding recurrent dropout to a 32-unit LSTM brought validation MAE down to 2.291 °C, 6.3% better than the baseline and close to the 2.27 °C from class. The stacked GRU was the best of the four at 2.253 °C.

Method 1: the number of units barely matters. The four stacked GRUs ended within 0.03 °C of each other: 2.226 (64/32), 2.250 (64/64), 2.253 (32/32) and 2.255 °C (16/16). That spread is about the same as the run-to-run noise I measured (Section 5), so I would not call any one of them better. What did change was how closely they fit the training data. The 64/64 model's training MAE was 2.359 °C against 2.913 °C for 16/16 (both measured with dropout on), yet they scored the same on validation. The extra capacity went into the training years, not into better forecasts, and the 16/16 model gets the same result with 3,185 parameters instead of 40,385.

Method 2: the GRU beat the LSTM. At the same sizes the LSTM was worse: 2.323 against 2.253 °C at 32/32, and 2.329 against 2.226 °C at 64/32. Those gaps (0.07 and 0.10 °C) are two to three times the noise, so I think this difference is real. Figure 1 shows the pattern. The LSTM 64/32 reached its best validation MAE at epoch 2, and after that it got worse while its training error kept falling. It was also the slowest model, at 92 seconds an epoch against 59 for the GRU of the same size. The LSTM's extra gate and memory track add weights, and on this data the simpler GRU seems to be enough.

Method 3: faster, but not more accurate. The Conv1D + GRU scored 2.297 °C and the Conv1D + LSTM 2.292 °C, both about 6% better than the baseline. That is a big step up from the convolution-only model in class, which scored around 2.9 °C because pooling throws away the order of the readings. Here the recurrent layer reads the pooled sequence in order. With half as many steps and one recurrent layer, an epoch took about 25 seconds, against 56–61 for the stacked GRUs. But both hybrids peaked at epoch 2 and then got worse (Figure 1), so they never caught up with the stacked GRU. The convolution layers have no dropout, and I think that is why they overfit so quickly.

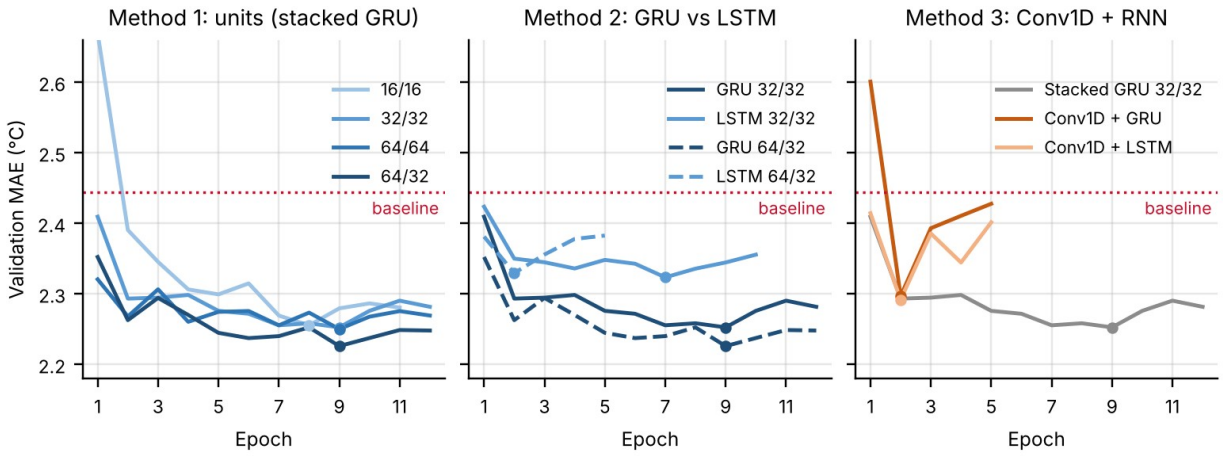


Figure 1. Validation MAE per epoch for the three methods. Dots mark each model's best epoch, which is the one kept. The dotted red line is the baseline (2.444 °C). The four GRU sizes overlap (left), the LSTMs sit above the GRUs (middle), and the Conv1D hybrids peak at epoch 2 (right).

The test set confirms the stacked GRU. My rule picked the three models with the lowest validation MAE, and all three were stacked GRUs: 64/32, 64/64 and 32/32. On the 2015–2016 test years they scored 2.413, 2.374 and 2.401 °C. The baseline scored 2.622 °C, so all three are 0.21 to 0.25 °C better, an improvement of 8.0 to 9.5%. The 32/32 model's 2.401 °C is very close to the 2.39 °C the course reports for the same model. The order changed between validation and test (64/64 came first on test and 64/32 last), but the spread is only 0.04 °C, so I treat them as tied here too.

Test errors are higher, but not because of overfitting to validation. Every model did worse on the test years than on the validation years, by 0.12 to 0.19 °C. The baseline, which learns nothing, also got worse by

0.18 °C. So the 2015–2016 period is simply harder to forecast, and the gap does not mean the models were tuned too closely to the validation data.

3. Results at a Glance

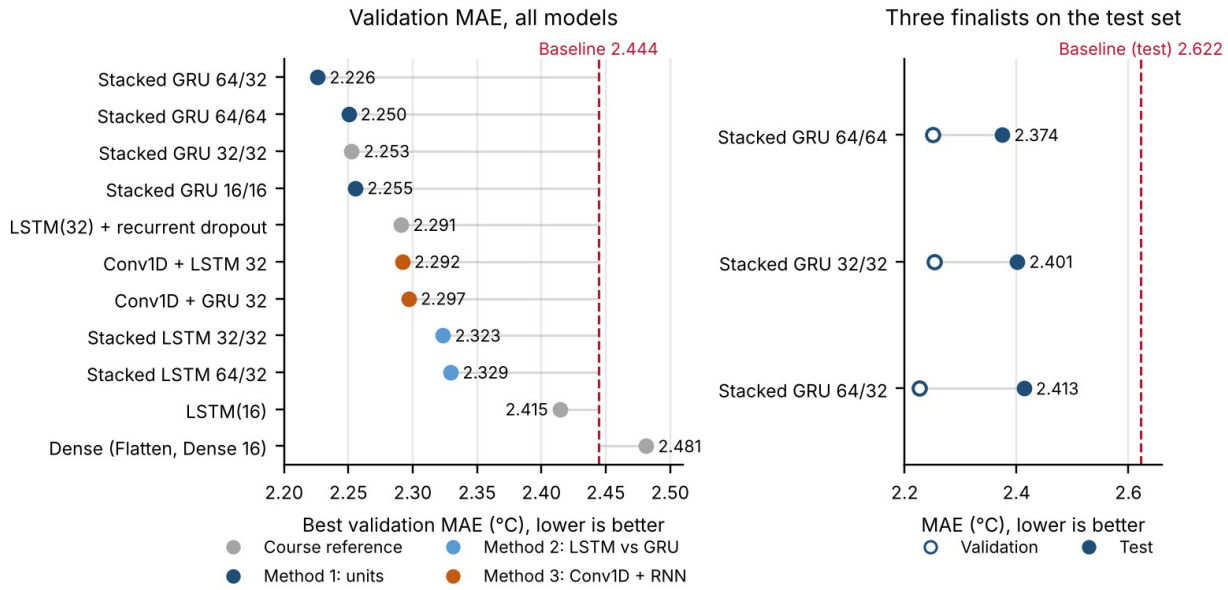


Figure 2. Left: the best validation MAE of every model, coloured by method, with the baseline as the red line. Right: the three finalists on the test set, validation (open circle) against test (filled). All three beat the baseline on test by 0.21–0.25 °C.

Model	Method	Parameters	Validation MAE (°C)	vs. baseline	Test MAE (°C)	Best / run epochs	Minutes
Common-sense baseline	Baseline	0	2.444	–	2.622	–	–
Dense (Flatten, Dense 16)	Course	26,913	2.481	−1.5%	–	2 / 5	0.3
LSTM(16)	Course	2,001	2.415	+1.2%	–	7 / 10	1.3
LSTM(32) + recurrent dropout	Course	6,049	2.291	+6.3%	–	7 / 10	4.8
Stacked GRU 32/32	Course	10,977	2.253	+7.8%	2.401	9 / 12	11.3
Stacked GRU 16/16	Method 1	3,185	2.255	+7.7%	–	8 / 11	10.6
Stacked GRU 64/64	Method 1	40,385	2.250	+7.9%	2.374	9 / 12	12.1
Stacked GRU 64/32	Method 1	24,801	2.226	+8.9%	2.413	9 / 12	11.8
Stacked LSTM 32/32	Method 2	14,369	2.323	+4.9%	–	7 / 10	10.1
Stacked LSTM 64/32	Method 2	32,673	2.329	+4.7%	–	2 / 5	7.7
Conv1D + GRU 32	Method 3	15,265	2.297	+6.0%	–	2 / 5	2.1
Conv1D + LSTM 32	Method 3	17,249	2.292	+6.2%	–	2 / 5	2.1

Table 3. Results for the baseline and all 11 models. "vs. baseline" is the improvement in validation MAE over the baseline's 2.444 °C (a minus sign means worse). Only the three models with the lowest validation MAE were run on the test set. Minutes is training time on the Kaggle T4 GPU. The shaded row is the recommended model.

4. Recommendation

I recommend the stacked GRU with 64 units in the first layer and 32 in the second (model 7). It had the lowest validation MAE of all 11 models (2.226 °C), and validation MAE is what the assignment asks us to choose on. On the test years it forecast the temperature 24 hours ahead with an average error of 2.41 °C, 8.0% better than assuming tomorrow will be like today, and it trains in about 12 minutes on a T4 GPU.

I would not switch to the 64/64 model just because it had the best test score (2.374 °C). Picking a model by its test result turns the test set into a second validation set, and the number would then overstate how well the model does on new data. If a smaller model is wanted, the class's 32/32 GRU is just as good in practice. It is within the noise of the 64/32 on both validation and test and has less than half the parameters (10,977 against 24,801). Based on these runs, the one thing I would avoid is the LSTM, which was worse at both sizes and slower to train.

5. Limitations and Next Steps

Each model was trained once, with one seed. I did train the four course models three times while building the notebook (twice on Colab and once on Kaggle, with small pipeline changes in between). Their validation MAE moved by up to 0.02–0.03 °C between runs for the recurrent models (the stacked GRU 32/32 scored 2.239, 2.233 and 2.253 °C), and by 0.06 °C for the dense model. So any difference below about 0.03 °C in this report could be noise. That covers all of Method 1 and the order of the three finalists on test.

Early stopping with a patience of 3 epochs is fairly strict. The LSTM 64/32 and both Conv1D hybrids stopped after only 5 epochs. Their validation error was already rising, so more epochs alone probably would not have helped, but I did not try other fixes such as dropout on the convolution layers or a lower learning rate. I also kept the class settings for the dropout rates, learning rate, 5-day window and hourly sampling, and only changed what the three methods ask for.

There are a few smaller points about the data and the process. The five gaps in the record were kept, so a few windows span uneven time steps, and the longest gap (3 days in October 2016) falls in the test period. The LSTM size in Method 2 was chosen from the Method 1 validation results. That is fair, because it uses validation data and not test data, but it means the size was not fixed in advance. And the work moved from Colab to Kaggle. The GPU and TensorFlow version were the same, but the numbers still shift slightly from one machine to another.

My next steps would be to run three seeds per model so the small differences can be tested properly, to add dropout to the convolution layers of the hybrids, and to try the feature engineering from the TensorFlow tutorial, such as time-of-day and time-of-year signals and wind as a vector instead of a speed and a direction.

6. Conclusion

Recurrent networks clearly beat the common-sense forecast on this data, but only once recurrent dropout is added; without it, the class models overfit after a couple of epochs. None of the three improvements beat the stacked GRU from class by more than the noise. More or fewer units made no real difference, the LSTM was worse, and adding convolutions made training faster but less accurate. The stacked GRU forecast the temperature a day ahead about 0.2 °C better than the baseline on the test years. My main takeaway is that on this problem, regularizing the model properly mattered more than the choice of architecture, and that small differences need repeat runs before they mean anything.

References

1. Max Planck Institute for Biogeochemistry. Weather Station Jena: Jena Climate data, 2009–2016. https://www.bgc-jena.mpg.de/wetter/ (copy used: https://storage.googleapis.com/tensorflow/tf-keras-datasets/jena_climate_2009_2016.csv.zip)
2. Chollet, F. (2021). Deep Learning with Python (2nd ed.), Chapter 10: Deep learning for timeseries. Manning.
3. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735
4. Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. In EMNLP 2014. arXiv:1406.1078. https://arxiv.org/abs/1406.1078
5. Gal, Y., & Ghahramani, Z. (2016). A theoretically grounded application of dropout in recurrent neural networks. In Advances in Neural Information Processing Systems 29. arXiv:1512.05287. https://arxiv.org/abs/1512.05287
6. Keras. LSTM and GRU layers: recurrent_dropout, unroll and the cuDNN requirements. https://keras.io/api/layers/recurrent_layers/lstm/ and https://keras.io/api/layers/recurrent_layers/gru/
7. Keras. timeseries_dataset_from_array. https://keras.io/api/data_loading/timeseries/
8. TensorFlow. Time series forecasting tutorial. https://www.tensorflow.org/tutorials/structured_data/time_series
9. Course materials, Advanced Machine Learning: slides and transcripts on deep learning for timeseries, recurrent neural networks, LSTM and GRU, and advanced uses of RNNs.

AI tools (Claude) helped draft and debug the notebook code and this report. I ran every cell, checked the outputs against the code, and am responsible for the results and conclusions in this report.